Web 2 – Séance 08

Props et children

Le problème

En séance 07, `RecipeCard` affiche **toujours la même recette** : les données sont écrites dans le composant.

```
const recipe = { title: "Pâtes Carbonara", imageUrl: "...", duration: 20, difficulty: "Facile" };
const RecipeCard = () => {
  return <div className="recipe-card">
    <h2>{recipe.title}</h2>
    <img src={recipe.imageUrl} alt={recipe.title} />
    <p>Durée : {recipe.duration} minutes</p>
    <p>Difficulté : {recipe.difficulty}</p>
  </div>;
};
```

Pour afficher 3 recettes différentes, il faudrait 3 composants presque identiques.

→ Il faut pouvoir **donner des données** au composant, comme on donne des arguments à une fonction.

Props : les paramètres d'un composant

- **Props** = *properties* → données passées par le composant **parent** au composant **enfant**
- Elles s'écrivent comme des attributs HTML dans le JSX du parent
- Le composant enfant les reçoit dans **un seul objet**, son premier paramètre

```
// Parent : on "appelle" le composant avec des props
<RecipeCard title="Pâtes Carbonara" imageUrl="..." duration={20} difficulty="Facile" />

// Enfant : un objet { title: "Pâtes Carbonara", imageUrl: "...", duration: 20, difficulty: "Facile" }
const RecipeCard = (props: { title: string; imageUrl: string; duration: number; difficulty: string }) => {
  return <h2>{props.title} ({props.duration} min)</h2>;
};
```

Ce que fait vraiment le JSX

Le JSX n'est pas du HTML : Vite le transforme en **appels de fonction**.

```
// Ce que vous écrivez
<RecipeCard title="Pâtes Carbonara" imageUrl="..." duration={20} difficulty="Facile" />

// Ce que le navigateur exécute (simplifié)
jsx(RecipeCard, { title: "Pâtes Carbonara", imageUrl: "...", duration: 20, difficulty: "Facile" });
```

- React appelle ensuite la fonction composant `RecipeCard` avec l'objet correspondant en argument
- Chaque attribut devient une propriété de l'objet `props`

Valeurs des props : chaînes et expressions

```jsx
<RecipeCard
  title="Pâtes Carbonara"      // chaîne de caractères : guillemets
  duration={20}                // nombre : accolades
  isVegetarian={false}         // booléen : accolades
  tags={["italien", "pâtes"]}  // tableau : accolades
  imageUrl={recipe.imageUrl}   // expression TypeScript : accolades
/>
```

- Entre guillemets → toujours une `string`
- Entre accolades → n'importe quelle expression TypeScript
- `duration="20"` passe la chaîne `"20"`, pas le nombre `20` : TypeScript signalera l'erreur si la prop est typée `number`

Typer les props avec une interface

Une interface décrit la forme de l'objet `props` : TypeScript vérifie chaque utilisation du composant.

```tsx
interface RecipeCardProps {
  title: string;
  imageUrl: string;
  duration: number;
  difficulty: number; // de 1 (facile) à 5 (difficile)
}

const RecipeCard = (props: RecipeCardProps) => {
  return (
    <div className="recipe-card">
      <h2>{props.title}</h2>
      <img src={props.imageUrl} alt={props.title} />
      <p>Durée : {props.duration} minutes</p>
      <p>Difficulté : {props.difficulty}/5</p>
    </div>
  );
};
```

- Convention : `NomDuComposantProps`
- `difficulty` devient un **nombre** de 1 à 5, comme dans les données du backend

Destructuring des props

```tsx
// ❌ Avant : props.xxx à chaque utilisation
const RecipeCard = (props: RecipeCardProps) => {
  return <div className="recipe-card">
    <h2>{props.title}</h2>
    <img src={props.imageUrl} alt={props.title} />
    <p>Durée : {props.duration} minutes</p>
    <p>Difficulté : {props.difficulty}/5</p>
  </div>;
};

// ✅ Après : destructuring dans la signature
const RecipeCard = ({ title, imageUrl, duration, difficulty }: RecipeCardProps) => {
  return <div className="recipe-card">
    <h2>{title}</h2>
    <img src={imageUrl} alt={title} />
    <p>Durée : {duration} minutes</p>
    <p>Difficulté : {difficulty}/5</p>
  </div>;
};
```

- La signature montre directement les props utilisées par le composant
- C'est l'écriture la plus courante dans la documentation React

Props optionnelles et valeurs par défaut

```tsx
interface RecipeCardProps {
  title: string;
  duration: number;
  description?: string; // optionnelle : string | undefined
  servings?: number;
}

const RecipeCard = ({
  title,
  duration,
  description = "Pas de description", // valeur par défaut si absente
  servings = 4,
}: RecipeCardProps) => {
  return (
    <div>
      <h2>{title}</h2>
      <p>{description}</p>
      <p>{duration} min, pour {servings} personnes</p>
    </div>
  );
};

<RecipeCard title="Guacamole" duration={10} />; // description et servings par défaut
```

Affichage conditionnel

Sans valeur par défaut, on peut n'afficher un élément **que si** la prop est fournie :

```
const RecipeCard = ({ title, duration, description }: RecipeCardProps) ⇒ {
  return (
    <div>
      <h2>{title}</h2>
      {description && <p>{description}</p>}
      <p>{duration} min</p>
    </div>
  );
};
```

- Si `description` vaut `undefined` (ou `""`), `&&` s'arrête et retourne cette valeur : React n'affiche rien
- Sinon, l'expression vaut le JSX situé après `&&`

Utiliser un composant plusieurs fois

Le même composant, appelé avec des props différentes, affiche des données différentes.

```
const App = () => {
  return (
    <div className="app">
      <h1>MiamMiam</h1>
      <RecipeCard
        title="Pâtes Carbonara"
        imageUrl="https://images.unsplash.com/photo-1612874742237-6526221588e3?w=800"
        duration={25}
        difficulty={2}
      />
      <RecipeCard
        title="Mousse au chocolat"
        imageUrl="https://images.unsplash.com/photo-1541783245831-57d6fb0926d3?w=800"
        duration={20}
        difficulty={3}
      />
    </div>
  );
};
```

Chaque `<RecipeCard />` est une **instance** indépendante du composant.

Les props sont en lecture seule

Un composant ne doit **jamais modifier** ses props.

```tsx
// ❌ Mauvais : modifier une prop
const RecipeCard = ({ title }: RecipeCardProps) => {
  title = title.toUpperCase();
  return <h2>{title}</h2>;
};

// ✅ Bon : calculer une nouvelle valeur dans une variable locale
const RecipeCard = ({ title }: RecipeCardProps) => {
  const displayTitle = title.toUpperCase();
  return <h2>{displayTitle}</h2>;
};
```

- Les props appartiennent au **parent** : c'est lui qui décide de leur valeur
- Un composant React doit se comporter comme une **fonction pure** : mêmes props → même affichage
- Pour des données qui changent dans le temps (clic, saisie…), on utilisera l'**état** (séance 11)

children : le contenu entre les balises

Tout ce qui est écrit **entre** la balise ouvrante et la balise fermante d'un composant est reçu dans une prop spéciale : `children`.

```tsx
import type { ReactNode } from "react";

interface CardProps {
  children: ReactNode;
}

const Card = ({ children }: CardProps) => {
  return <div className="card">{children}</div>;
};

// Utilisation
<Card>
  <h2>Pâtes Carbonara</h2>
  <p>Un classique italien.</p>
</Card>
```

- `ReactNode` = tout ce que React sait afficher : JSX, texte, nombre, `null`, `undefined`…

Pourquoi children ?

Permet de créer des composants **conteneurs** : ils définissent un cadre, le parent décide du contenu.

```tsx
interface PageLayoutProps {
  title: string;
  children: ReactNode;
}

const PageLayout = ({ title, children }: PageLayoutProps) => {
  return <div className="page">
    <header><h1>{title}</h1></header>
    <main>{children}</main>
    <footer>&copy; 2026 MiamMiam</footer>
  </div>;
};

const App = () => (
  <PageLayout title="MiamMiam">
    <RecipeCard title="Guacamole" imageUrl="..." duration={10} difficulty={1} />
  </PageLayout>
);
```

`PageLayout` ne sait pas ce qu'il affiche : il est réutilisable pour toutes les pages.

Composition de composants

Une application React est un **arbre de composants** : chaque composant en utilise d'autres.

```
App
└── PageLayout
    └── RecipeList
        ├── RecipeCard (Pâtes Carbonara)
        ├── RecipeCard (Mousse au chocolat)
        └── RecipeCard (Guacamole)
```

- Les données descendent de parent à enfant, **jamais l'inverse**
- Chaque composant a une responsabilité : mise en page, liste, carte…

Découper en composants : quand ?

Pas de règle absolue, mais quelques critères :

- **Répétition** : le même morceau de JSX apparaît plusieurs fois → un composant avec des props
- **Responsabilité** : un bloc a un rôle clair (en-tête, carte, formulaire) → un composant nommé d'après ce rôle
- **Lisibilité** : un composant dépasse une cinquantaine de lignes de JSX → le découper
- **Cadre réutilisable** : une mise en page commune à plusieurs contenus → un composant avec `children`

Trop découper est aussi un défaut : un composant de 3 lignes utilisé une seule fois n'apporte souvent rien.

Récapitulatif Séance 08

- **Props** – Données passées par le parent à l'enfant, reçues dans un objet
- **JSX** – `<RecipeCard title="..." />` devient un appel `RecipeCard({ title: "..." })`
- **Interface de props** – Typage des props, erreurs détectées dans VSCode
- **Destructuring** – `({ title, duration }: RecipeCardProps)` dans la signature
- **Props optionnelles** – `?` dans l'interface et valeur par défaut dans le destructuring
- **Lecture seule** – Un composant ne modifie jamais ses props
- **children** – Contenu placé entre les balises, typé `ReactNode`
- **Composition** – Une application est un arbre de composants

Prochaine séance : Séance 09 – Assets, CSS et MUI

Exercice filé S08

1. Dans `RecipeCard`, remplacez l'objet `recipe` par des props typées avec une interface `RecipeCardProps` : `title`, `imageUrl`, `duration` et `difficulty` (nombre de 1 à 5)
2. Ajoutez une prop optionnelle `description`, affichée seulement si elle est fournie
3. Créez un composant `RecipeList` dans `src/components` qui affiche 3 `RecipeCard` avec des recettes différentes (reprenez celles du backend : `data/recipes.json`)
4. Créez un composant `PageLayout` avec une prop `title` et des `children` : en-tête avec le titre, `<main>` avec le contenu, pied de page
5. Dans `App.tsx`, affichez `RecipeList` à l'intérieur de `PageLayout`
6. Vérifiez que la console du navigateur n'affiche aucun warning

Exercice complémentaire EC04

La page HTML de la séance 08 (sur moodle) est la carte d'un restaurant : beaucoup de blocs s'y répètent, avec de petites différences.

1. Créez un projet `EC04` et copiez le CSS de la page dans `src/index.css`
2. Sur papier, pour chaque bloc qui se répète, notez :
   - ce qui change d'une occurrence à l'autre → des **props**
   - ce qui est parfois absent → des **props optionnelles** et un **affichage conditionnel**
   - les cadres qui entourent un contenu différent à chaque fois → des **children**
3. Créez les composants et reproduisez la page à l'identique : le JSX d'un bloc répété n'apparaît **qu'une seule fois**, dans son composant